

CO₂ - AT₃ - SCENARIO BASED
TASK

1. Scenario : Parsing Table Conflict in LL(1)

In an LL(1) parser, the parsing table is constructed using the first and follow sets of the grammar. The first set determines the terminals that can appear at the beginning of strings derived from a production while the follow set contains terminals that can appear immediately after a non-terminal.

A conflict occurs when more than one production is entered into the same table cell for a non-terminal and an input symbol, because the grammar contains left recursion, common prefixes or is ambiguous.

Since LL(1) parsing is deterministic, every table entry must contain one production

Multiple entries make the parser non-deterministic and may lead to incorrect parsing or parser failure

Decision: The grammar should be modified by eliminating left recursion, performing left factoring, and ensuring that the FIRST and FOLLOW sets do not overlap. This grammar is converted into a valid LL(1) grammar with a conflict-free parsing table.

2. Scenario Incorrect Parse Tree Generation

The syntax analyzer constructs a parse tree by applying grammar production according to the input tokens for the expression $id + id - id$, the parse tree determines the order in which operations are evaluated.

If the grammar does not specify operator associated the expression may be grouped incorrectly. for example, it may be interpreted as $id + (id + id)$ instead of the correct $(id + id) - id$. This results in incorrect evaluation because the operators are

not associated properly.

Grammar rules directly influence the structure of the parse tree. A grammar that does not enforce associativity allows multiple valid parse trees for the same expression.

Decision: The grammar should enforce left associativity for addition subtraction. for example

- $E \rightarrow E + E / E - T / T$
- $T \rightarrow id$

This ensures expressions are evaluated from left to right, producing the correct parse tree and accurate results.

3. Scenario : Invalid Expression Handling.

The lexical analyzer correctly identifies the tokens in the expression $a + * b$ as identifiers and operators. However, the syntax analyzer checks whether these tokens follow the grammar rules.

The error occurs because the grammar expects an operand after the '+' operator, but another operator '*' appears instead. Therefore, the token

sequence violated the grammar, making it impossible to construct a valid parse tree.

In predictive parsing (LL), the parser detects the error when no valid production exists in the parsing table for the current input symbol. In shift-reduce parsing (LR), the error when no valid shift or reduce action is available.

Without proper recovery, the parser stops after reporting the error.

Decision: The parser should implement error detection and recovery techniques, such as panic-mode recovery, phrase-level recovery, or synchronization using FOLLOW sets. These methods allow the parser to continue analyzing the remaining input while providing meaningful error message.

4. Scenario: Ambiguity in Conditional Statements

The grammar

- $S \rightarrow \text{if } E \text{ then } S$
- $S \rightarrow \text{if } E \text{ then } S \text{ else } S$
- $S \rightarrow a$

is ambiguous because the parser cannot determine if statement an else belongs to. This is known as the dangling else problem.

When constructing the parsing table, conflicts arise because both productions begin with the same prefix: $\text{if } E \text{ then } S$.

The parser cannot decide whether to reduce using the first production or continue parsing the second production containing else.

This ambiguity affects deterministic parsing by introducing parsing conflicts and producing multiple possible parse trees.

5. Scenario : Operator Precedence Conflict

The grammar

- $E \rightarrow E + E$
- $E \rightarrow E * E$
- $E \rightarrow \text{id}$

is ambiguous because not define operator precedence or associativity.

for the input $\text{id} + \text{id} * \text{id}$, two different parse trees are possible:

- $(id + id) * id$
- $(id * id) + id$

The correct evaluation requires multiplication to have higher precedence than addition. Since the grammar does not specify this rule, the parser cannot determine the correct parse tree.

Operator precedence and associativity are essential to ensure expressions are evaluated correctly.

Decision: The grammar should be rewritten to enforce precedence. for example:

- $E \rightarrow E + T / T$
- $T \rightarrow E * F / F$
- $F \rightarrow id$

Here :-

- Multiplication has higher precedence than addition
- Addition and Multiplication are left associative.

Alternatively parser generator can use precedence and associativity to resolve such conflicts automatically.